CS341 ExpZ GPT Designs

Tomas Salaz

September 29, 2025

1 Design 1

```
    FUNCTION greatestBitPos(x):
    # Return mask with only the most-significant 1-bit set; 0 if x == 0
    y ← x
    y ← y OR (y >> 1)
    y ← y OR (y >> 2)
    y ← y OR (y >> 4)
    y ← y OR (y >> 8)
    y ← y OR (y >> 16)
    RETURN y AND NOT (y >> 1)
END

FUNCTION howManyBits(x):
    # Minimum bits to represent x in 2's complement
    sign ← x >> 31                  # all 1s if x < 0 else 0
    a ← x XOR sign                  # if x<0 use ~x, else x
    count ← 1                       # include sign bit at the end

    b ← 0
    IF (a >> 16) != 0: b ← b + 16; a ← a >> 16
    IF (a >> 8)  != 0: b ← b + 8;  a ← a >> 8
    IF (a >> 4)  != 0: b ← b + 4;  a ← a >> 4
    IF (a >> 2)  != 0: b ← b + 2;  a ← a >> 2
    IF (a >> 1)  != 0: b ← b + 1;  a ← a >> 1
    # a is now 0 or 1
    RETURN b + a + count
END

FUNCTION satAdd(x, y):
    # Saturating x + y to INT_MAX/INT_MIN on overflow
    sum ← x + y
    sx ← x >> 31          # sign bits
    sy ← y >> 31
    ss ← sum >> 31
```

```
    posOverflow ← (NOT sx) AND (NOT sy) AND ss
    negOverflow ← sx AND sy AND (NOT ss)
    overflow   ← posOverflow OR negOverflow

    TMIN ← (1 << 31)
    TMAX ← NOT TMIN

    IF overflow == 0:
        RETURN sum
    ELSE IF posOverflow != 0:
        RETURN TMAX
    ELSE:
        RETURN TMIN
END

FUNCTION float_i2f(x):
    # Return IEEE-754 single-precision bits for (float)x
    IF x == 0: RETURN 0

    IF x < 0:
        signBit ← 1
        ax ← unsigned_abs(x)   # two's-complement absolute value
    ELSE:
        signBit ← 0
        ax ← x (as unsigned)

    # Find p = index of highest set bit in ax (0..31)
    p ← 31
    WHILE ((ax >> p) AND 1) == 0: p ← p - 1

    exp ← p + 127               # unbiased exponent
    IF p  23:
        frac ← (ax << (23 - p)) AND 0x7FFFFF
    ELSE:
        shift ← p - 23
        base  ← ax >> shift                 # includes leading 1
        frac  ← base AND 0x7FFFFF           # drop implicit 1

        rem   ← ax AND ((1 << shift) - 1)   # bits shifted out
        guard ← (rem >> (shift - 1)) AND 1
        sticky← (rem AND ((1 << (shift - 1)) - 1)) != 0
        roundUp ← (guard == 1) AND (sticky OR (frac AND 1) == 1)

        IF roundUp:
            frac ← frac + 1
```

```
            IF frac == (1 << 23):           # carry into hidden bit
                frac ← 0
                exp ← exp + 1

    RETURN (signBit << 31) OR (exp << 23) OR frac
END

FUNCTION trueFiveEighths(x):
    # Compute trunc_toward_zero( (5 * x) / 8 ) without overflow
    # Decompose x = 8*q_floor + r, where q_floor = floor(x/8), r in [0..7]
    q_floor ← x >> 3
    r       ← x AND 7
    neg     ← (x < 0) ? 1 : 0

    # Convert floor-quotient to trunc-toward-zero quotient
    # If x < 0 and r != 0, then q_trunc = q_floor + 1, else q_trunc = q_floor
    add1    ← (neg == 1 AND r != 0) ? 1 : 0
    q_trunc ← q_floor + add1

    # Corresponding remainder for trunc division:
    # If added 1, remainder becomes r - 8 (negative); else r
    r_trunc ← (add1 == 1) ? (r - 8) : r      # r_trunc in [-7..7]

    # Now (5*x)/8 = 5*q_trunc + trunc_toward_zero( (5 * r_trunc)/8 )
    partQ ← (q_trunc << 2) + q_trunc         # 5 * q_trunc

    t ← (r_trunc << 2) + r_trunc             # 5 * r_trunc (fits small range)
    # trunc_toward_zero divide-by-8 using bias for negatives:
    bias ← ((t >> 31) AND 7)                  # 7 if t < 0 else 0
    partR ← (t + bias) >> 3

    RETURN partQ + partR
END

FUNCTION trueThreeFourths(x):
    # Compute trunc_toward_zero( (3 * x) / 4 ) without overflow
    # Decompose x = 4*q_floor + r, where q_floor = floor(x/4), r in [0..3]
    q_floor ← x >> 2
    r       ← x AND 3
    neg     ← (x < 0) ? 1 : 0

    # Adjust to trunc-toward-zero quotient
    add1    ← (neg == 1 AND r != 0) ? 1 : 0
    q_trunc ← q_floor + add1

    # Adjust remainder accordingly (becomes negative if we added 1)
```

```
    r_trunc ← (add1 == 1) ? (r - 4) : r      # r_trunc in [-3..3]

    # (3*x)/4 = 3*q_trunc + trunc_toward_zero( (3 * r_trunc)/4 )
    partQ ← (q_trunc << 1) + q_trunc         # 3 * q_trunc

    t ← (r_trunc << 1) + r_trunc             # 3 * r_trunc
    # trunc_toward_zero divide-by-4 using bias for negatives:
    bias ← ((t >> 31) AND 3)                  # 3 if t < 0 else 0
    partR ← (t + bias) >> 2

    RETURN partQ + partR
END
```